BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BẢN GIẢI THAM KHẢO HOÀN CHỈNH

ĐỀ TÀI DT076

TÌM VÀ IN ĐƯỜNG ĐI ÍT CẠNH NHẤT BẰNG BFS

Từ đặc tả đến mã nguồn, kiểm thử và chạy lại

Giảng viên học phần	Hồ Nhựt Minh
Học phần / nhóm	INT13185 / 03
Tính chất tài liệu	Lời giải mẫu dùng giảng dạy
Ngày kiểm chứng	17/10/2026

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BẢN GIẢI THAM KHẢO HOÀN CHỈNH

ĐỀ TÀI DT076

TÌM VÀ IN ĐƯỜNG ĐI ÍT CẠNH NHẤT BẰNG BFS

Từ đặc tả đến mã nguồn, kiểm thử và chạy lại

Giảng viên học phần	Hồ Nhựt Minh
Học phần / nhóm	INT13185 / 03
Tính chất tài liệu	Lời giải mẫu dùng giảng dạy
Ngày kiểm chứng	17/10/2026
Phiên bản nguồn C	af228fa7ea70

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI THAM KHẢO

DT076 — Tìm và in đường đi ít cạnh nhất bằng BFS

Phạm vi: đồ án học phần; trình bày theo mẫu báo cáo đã cung cấp. Đây không phải xác nhận quy chuẩn đồ án tốt nghiệp cấp Học viện.

Dữ liệu vào và kết quả ra

Đồ thị đơn, vô hướng, không trọng số G=(V,E); V={1,…,n}, 1≤n≤100, 0≤m≤n(n−1)/2. Các cạnh là cặp số nguyên trong miền; cấm cạnh khuyên và cạnh trùng, kể cả cặp đảo. Nguồn s và đích t thuộc V. Nhập bàn phím hoặc khai báo trực tiếp trong chương trình.

In thứ tự lấy đỉnh khỏi hàng BFS, d[v], cha[v], một đường đi ít cạnh nhất từ s đến t và số cạnh. Danh sách kề xét tăng dần; cha chỉ gán ở lần đầu phát hiện. Không tới được thì d[t]=−1 và đường rỗng. Nếu s=t, trả [s] với 0 cạnh. BFS vẫn duyệt hết thành phần chứa s để bảng d, cha có cùng nghĩa ở mọi ca.

Ví dụ	Đầu vào	Kết quả
1	n=4; E={(1,2),(1,3),(2,4),(3,4)}; s=1; t=4	Thứ tự 1,2,3,4; đường 1→2→4; 2 cạnh
2	n=3; E={(1,2)}; s=1; t=3	Thứ tự 1,2; KHÔNG CÓ ĐƯỜNG; d[3]=−1

Sản phẩm đã giải trong bản mẫu

Tự dựng danh sách kề và FIFO bằng mảng; không dùng thư viện tìm đường thay BFS.

Có đặc tả hàm, mã giả, bảng truy vết, chứng minh đúng và phân tích chi phí toàn quy trình.

Có 12 ca chức năng và 16 ca validator/hàm phụ, tổng 59 phép assert; bảng expected/actual có mã ca.

Đối chiếu Floyd–Warshall trên 1.099 đồ thị nhỏ và 26.705 cặp nguồn/đích; khảo sát số thao tác tại 9 cấu hình.

Có 12 hình từ log thực chạy; Git commit và clone cục bộ có thật; mã đầy đủ cùng lệnh chạy lại.

Căn cứ: giáo trình INT1306, mục 6.2, 6.3.2, trang in 140–145 (PDF 152–157); Bài tập 28 trang in 159 (PDF 171). Đồ thị hình thoi do bản mẫu tự thiết kế.

PHIẾU CHẤM VÀ CÁCH ĐỐI CHIẾU

Tiêu chí	Tối đa /10	Vị trí minh chứng	Điểm chấm
Bài toán, đầu vào và đầu ra	1.5	Ch.1; hai ví dụ; hợp đồng s=t, không có đường	.....
Cấu trúc dữ liệu tự cài	2	Ch.2–3; danh sách kề; FIFO dau/cuoi	.....
Thuật toán và truy vết	2.5	Ch.2–3; bất biến; bảng d/cha; Phụ lục A	.....
Kiểm thử và độ phức tạp	2	Ch.4; 28 ca; oracle độc lập; 9 khảo sát	.....
Báo cáo, minh chứng, GitHub	1	12 hình và commit/clone local; URL GitHub do SV cung cấp khi nộp	.....
Bảo vệ và giải thích	1	Ch.5; chạy ca mới; giải thích lần đầu phát hiện	.....

Người chấm điền điểm theo sản phẩm và phần bảo vệ thực tế. Không coi bản tham khảo này là bài của sinh viên hoặc mặc nhiên đã đạt điểm tối đa. Điểm từng mục nằm trong [0; trọng số], cộng sáu mục được tối đa 10. Nếu nhập điểm thô trên thang 10 cho từng mục, nhân trọng số rồi chia 10.

Tổng điểm: ..... /10. Người chấm: .....

Chỉ dẫn minh chứng sinh viên thống nhất theo A01–A11 trong hướng dẫn Classroom. Tám ảnh thực thi bao gồm hai ví dụ, hai ca tự thiết kế, ba ca biên và một ca đầu vào sai; thêm ảnh tổng kết test, chạy từ clone và GitHub/commit. Bảng truy vết, đối chiếu và khảo sát có thể dùng chính các ca này, không yêu cầu một bộ 11 ảnh thứ hai.

NGUỒN GỐC VÀ CÁCH DÙNG BẢN GIẢI

Tài liệu này là bản giải giảng dạy hoàn chỉnh cho DT076. Không gán tên, MSSV, lời cam đoan hay chữ ký của một sinh viên giả định. Bìa và các phần trình bày kế thừa mẫu Word trong thư mục MAU TRINH BAY; phần nội dung được viết lại theo yêu cầu học phần.

Đã thực chạy Julia 1.13.1 và thử clone Git cục bộ từ commit C ghi trong báo cáo. Các hình là ảnh hiển thị nguyên văn log chương trình, không nhận là ảnh chụp terminal cá nhân của sinh viên. Log thô, mã, test và Git bundle được phát kèm để tái hiện.

GitHub cá nhân không được tạo hoặc công bố thay sinh viên. Vì vậy bản tham khảo sử dụng commit và clone local có thật, không điền một URL giả. Khi nộp, sinh viên xuất bản kho của mình, ghi URL/commit/permalink thật và thay bằng chứng cá nhân theo hướng dẫn.

Sinh viên đọc lời giải để hiểu cách đặc tả, lập luận và kiểm chứng; sau đó làm đúng mã đề được giao theo MSSV. Đề khác BFS phải thay bài toán, cấu trúc, thuật toán, ca thử và phân tích. Ghi rõ phần tham khảo; không nộp nguyên bản giải này dưới tên mình.

TÓM TẮT

Đồ án giải bài toán tìm một đường đi ít cạnh nhất giữa hai đỉnh của đồ thị vô hướng không trọng số bằng duyệt theo chiều rộng. Danh sách kề được chuẩn hóa tăng dần; hàng đợi mảng tự cài đảm bảo FIFO. Mỗi đỉnh được đánh dấu khi đưa vào hàng, nhận khoảng cách và cha một lần, rồi truy vết từ đích về nguồn.

Lời giải gồm chương trình Julia chỉ dùng thư viện chuẩn, chế độ nhập bàn phím, tám demo, 59 phép assert trên 28 ca có định danh, oracle Floyd–Warshall và khảo sát số phép toán. Tất cả phép kiểm đã chạy thành công tại commit C; clone sạch cục bộ chạy lại cùng quy trình với mã thoát 0.

BFS trên danh sách kề đã chuẩn bị có thời gian O(n+m), bộ nhớ phụ O(n). Chi phí chuẩn bị kề, sắp xếp và kiểm tuyến được tách riêng. Tập đồ thị $n \leq 5$ cho bằng chứng thực nghiệm hữu hạn; chứng minh bất biến FIFO đảm bảo tính đúng trên miền $n \leq 100$ đã đặc tả.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI THAM KHẢO i
Dữ liệu vào và kết quả ra i
Sản phẩm đã giải trong bản mẫu i
PHIẾU CHẤM VÀ CÁCH ĐỐI CHIẾU ii
NGUỒN GỐC VÀ CÁCH DÙNG BẢN GIẢI iii
TÓM TẮT iv
CHƯƠNG 1. PHÂN TÍCH BÀI TOÁN 1
1.1. Vấn đề cần giải và căn cứ giáo trình 1
1.2. Hợp đồng chính xác của đầu vào 1
1.3. Đầu ra, nhiều nghiệm và trường hợp đặc biệt 2
1.4. Hai ví dụ được giải tay 2
1.5. Tiêu chí nghiệm thu 3
CHƯƠNG 2. CƠ SỞ VÀ CHỨNG MINH 4
2.1. Danh sách kề và hàng đợi FIFO 4
2.2. Mã giả hoàn chỉnh 4
2.3. Bất biến và tính tối ưu 4
2.4. Tính đúng và kết thúc của truy vết 5
2.5. Độ phức tạp theo từng công đoạn 5
CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT 7
3.1. Kiến trúc tệp và luồng thực hiện 7
3.2. Hợp đồng năm hàm chính 7
3.3. Những dòng quyết định tính đúng 8
3.4. Truy vết từng bước ví dụ hình thoi 8
3.5. Nhập bàn phím và xử lý dữ liệu sai 9
CHƯƠNG 4. KIỂM THỬ VÀ KẾT QUẢ 10
4.1. Môi trường, phiên bản và phương pháp 10
4.2. Bảng kiểm 12 ca chức năng 10
4.3. Mười sáu ca kiểm helper và dữ liệu vào 11
4.4. Tám lần thực thi với kết quả cụ thể 13
4.5. Tổng kết test và đối chiếu độc lập 20
4.6. Khảo sát số thao tác theo quy mô 21
4.7. Chạy lại từ clone sạch và truy nguyên phiên bản 22
CHƯƠNG 5. ĐÁNH GIÁ VÀ TÁI HIỆN 24
5.1. Đối chiếu nhiệm vụ với sản phẩm 24
5.2. Giới hạn và hướng phát triển 24
5.3. Các câu hỏi bảo vệ đã có lời giải 24
Vì sao không đánh dấu lúc lấy khỏi hàng? 24
Vì sao cha[4] không đổi ở cạnh 3–4? 25
Đếm số cạnh hay số đỉnh trên tuyến? 25
BFS có dừng ngay khi thấy t không? 25
Kiểm từng cặp cạnh có chứng minh tối ưu không? 25
Chạy hết n≤5 có đảm bảo không lỗi mọi n không? 25
5.4. Thực hiện lại từ gói đã tải 26
TÀI LIỆU THAM KHẢO 27
PHỤ LỤC A. TOÀN BỘ MÃ LỖI 1
PHỤ LỤC B. DEMO VÀ NHẬP BÀN PHÍM 3
PHỤ LỤC C. CHẠY TỰ ĐỘNG VÀ TOÀN VẸN 5
PHỤ LỤC D. PHIẾU TỰ KIỂM TRƯỚC NỘP 6

DANH MỤC HÌNH

Hình 1.1. Đồ thị hình thoi dùng cho ví dụ 1 2
Hình 3.1. Quy trình chuẩn bị kề, BFS, truy vết và đối chiếu 7
Hình 4.1. Ảnh hiển thị log thực chạy T01 — hai tuyến cùng độ dài 13
Hình 4.2. Ảnh hiển thị log thực chạy T02 — đích không tới được 14
Hình 4.3. Ảnh hiển thị log thực chạy T03 — nguồn trùng đích 15
Hình 4.4. Ảnh hiển thị log thực chạy T07 — nguồn cô lập 16
Hình 4.5. Ảnh hiển thị log thực chạy T05 — chuỗi phải truy vết dài 17
Hình 4.6. Ảnh hiển thị log thực chạy T06 — chu trình 18
Hình 4.7. Ảnh hiển thị log thực chạy T08 — miền nhỏ nhất 19
Hình 4.8. Ảnh hiển thị log thực chạy V07 — cạnh vô hướng bị nhập lặp 20
Hình 4.9. Kết quả thực chạy 43+16=59 phép assert 20
Hình 4.10. Oracle Floyd–Warshall trên toàn bộ đồ thị đơn n≤5 21
Hình 4.11. Clone cục bộ mới tại C rồi chạy toàn bộ Run_all.ps1 22
Hình 4.12. Git commit nguồn C và trạng thái kho đã kiểm 23

DANH MỤC BẢNG

Bảng 1.1. Điều kiện đầu vào và phản ứng của chương trình.....1
Bảng 2.1. Chi phí; m là số cạnh vô hướng 5
Bảng 3.1. Cấu trúc mã nguồn phát hành 7
Bảng 3.2. Đối số, tiền điều kiện và giá trị trả về 7
Bảng 3.3. Trạng thái sau khi xử lý xong u; thứ tự đỉnh trong d và cha là 1..4 8
Bảng 4.1. Expected và actual đã chạy; ký hiệu d là số cạnh tới đích 10
Bảng 4.2. Ca V01–V16; actual tóm tắt sau khi predicate đã xác nhận 11
Bảng 4.3. Bộ đếm thực chạy; BFS duyệt hết thành phần, nguồn 1, đích n 21
Bảng 5.1. Kết quả nghiệm thu có thể kiểm lại 24

DANH MỤC KÝ HIỆU

Ký hiệu	Ý nghĩa
n, m	Số đỉnh, số cạnh vô hướng
ke[v]	Danh sách láng giềng tăng dần của v
d[v]	Số cạnh ít nhất từ s đến v; −1 nếu chưa tới
cha[v]	Đỉnh phát hiện v; 0 nếu không có cha
hang, dau, cuoi	Mảng FIFO và hai chỉ số đầu/cuối đang dùng
l, Δ	Số đỉnh trên tuyến; bậc lớn nhất của đồ thị
C	Commit mã dùng tạo kết quả; khác commit đóng gói tài liệu

CHƯƠNG 1. PHÂN TÍCH BÀI TOÁN

1.1. Vấn đề cần giải và căn cứ giáo trình

Với các liên kết có cùng chi phí một bước, cần tìm cách đi qua ít liên kết nhất. Đỉnh biểu diễn vị trí, cạnh biểu diễn khả năng chuyển trực tiếp; độ dài tuyến là số cạnh. Ví dụ minh họa chỉ dùng nhãn nguyên và các cặp cạnh, phù hợp các thuật toán cơ sở của giáo trình.

Mục 6.2 trình bày cách biểu diễn đồ thị; mục 6.3.2 trình bày BFS; Bài tập 28 yêu cầu đường đi ít cạnh nhất. Bản này triển khai thêm mảng cha để in tuyến, kiểm dữ liệu và đối chiếu độc lập. Phần bổ sung phục vụ kiểm chứng, không nhận là mã nguyên văn giáo trình.

Mã INT1306 trên giáo trình/đề cương và INT13185 trên danh sách lớp được giữ đúng nguồn. Nội dung được đối chiếu theo chương/mục, không suy diễn hai mã học phần tương đương về quản lý đào tạo.

1.2. Hợp đồng chính xác của đầu vào

Bảng 1.1. Điều kiện đầu vào và phản ứng của chương trình

Trường	Miền hợp lệ	Khi vi phạm
n	Số nguyên; $1 \le n \le 100$	ArgumentError nếu ngoài miền
Cạnh (u,v)	Nhãn nguyên 1..n; $u \ne v$	Từ chối nhãn sai hoặc cạnh khuyên
Tập cạnh	Mỗi cạnh vô hướng xuất hiện một lần	(1,2) và (2,1) là trùng
s,t	Số nguyên trong 1..n	ArgumentError nếu ngoài miền
Đầu vào bàn phím	n m; m dòng u v; một dòng s t	Sai số trường hoặc ký tự không nguyên gây lỗi nhập

Hàm lõi nhận các đối số đã có kiểu số nguyên theo chữ ký và cặp cạnh; không cam kết xử lý mọi đối tượng Julia tùy ý. Chế độ bàn phím parse số nguyên trước khi gọi lõi. Sai kiểu không thể parse được báo lỗi, không tự sửa hoặc bỏ dữ liệu.

Không cần CSV, Excel, cơ sở dữ liệu hay mạng để chạy thuật toán. Dữ liệu demo nằm trực tiếp trong demo.jl; dữ liệu của chế độ --nhap được gõ trong terminal. Tệp log,

Git bundle và báo cáo là sản phẩm phân phối/kiểm chứng, không phải dữ liệu vào thuật toán.

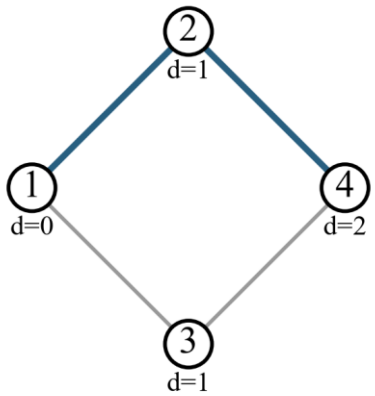
1.3. Đầu ra, nhiều nghiệm và trường hợp đặc biệt

Với một nguồn s, BFS trả d và cha cho toàn bộ thành phần liên thông chứa s, không dừng sớm khi gặp t. Quy ước này làm bộ đếm và bảng d/cha nhất quán. Trạng thái d[v]=−1 phân biệt đỉnh chưa đến được với nguồn d[s]=0.

Khi có nhiều tuyến ngắn nhất, thứ tự xét kề tăng dần và FIFO quyết định tuyến được lưu. Ở hình thoi, 1→2→4 và 1→3→4 đều dài 2, nhưng cha[4]=2 vì đỉnh 2 được xử lý trước 3. Không thay cha[4] khi xét cạnh 3–4.

Nếu s=t, đường chỉ có một đỉnh, 0 cạnh; vẫn có thể duyệt các đỉnh khác để tạo bảng d. Nếu đích không tới được, đường trả rỗng và so_canh=−1, in thông báo rõ. Không nhầm đường rỗng với tuyến hợp lệ có 0 cạnh.

1.4. Hai ví dụ được giải tay



Đường được chọn 1 → 2 → 4 có 2 cạnh

Hình 1.1. Đồ thị hình thoi dùng cho ví dụ 1

Ví dụ 1: từ 1 có hai láng giềng 2,3, nên d[2]=d[3]=1 và cha[2]=cha[3]=1. Lấy 2 khỏi hàng, phát hiện 4, gán d[4]=2 và cha[4]=2. Lấy 3, đỉnh 4 đã phát hiện nên giữ nguyên cha. Truy ngược 4,2,1 rồi đảo thành 1,2,4; hai cạnh đều thuộc E.

Ví dụ 2: n=3, E={(1,2)}, s=1, t=3. Sau khi xử lý 1 rồi 2, hàng rỗng; d=[0,1,−1], cha=[0,1,0]. Đỉnh 3 không thuộc thành phần nguồn nên kết luận không có đường. Không chạy truy_vet từ cha[3]=0 để tạo ra tuyến giả.

1.5. Tiêu chí nghiệm thu

Một kết quả đạt yêu cầu phải đúng số cạnh tối thiểu, có đầu/cuối đúng, mọi cặp đỉnh liên tiếp là cạnh thật, và dùng đúng quy tắc chọn khi hòa. Chỉ kiểm tuyến hợp lệ chưa đủ xác nhận tối ưu. Chỉ so ảnh giống mẫu cũng chưa đủ; cần test độc lập và giải thích bất biến.

Khả năng tái hiện được kiểm bằng clone một thư mục mới rồi chạy Run_all.ps1. Báo cáo, log và bảng kết quả cùng chỉ về commit C. Không lấy việc chương trình kết thúc không lỗi làm bằng chứng duy nhất cho tính đúng.

CHƯƠNG 2. CƠ SỞ VÀ CHỨNG MINH

2.1. Danh sách kề và hàng đợi FIFO

Đồ thị vô hướng lưu mỗi cạnh (u,v) vào ke[u] và ke[v], tổng 2m mục kề. So với ma trận n×n, danh sách kề phù hợp việc chỉ xét các cạnh hiện có. Đổi biểu diễn có thể đổi chi phí duyệt, nên độ phức tạp phải nêu cùng cách lưu.

Hàng đợi được cấp phát n phần tử. Ban đầu hang[1]=s, dau=cuoi=1. Lấy hang[dau] rồi tăng dau; thêm v bằng tăng cuoi rồi ghi hang[cuoi]. Đoạn còn chờ là dau:cuoi. Khi dau>cuoi, hàng rỗng; các ô cũ trong mảng không còn là phần tử đang chờ.

Mỗi đỉnh được đưa vào hàng tối đa một lần nên cuoi≤n. Không cần dịch mảng như popfirst! hoặc quay vòng trong một lượt BFS này. Dùng mảng cấp trước giúp mỗi lần enqueue/dequeue O(1).

2.2. Mã giả hoàn chỉnh

```
BFS_CHA(ke, s):
    d[1..n] = -1; cha[1..n] = 0
    hang[1] = s; dau = 1; cuoi = 1; d[s] = 0
    while dau <= cuoi:
        u = hang[dau]; dau = dau + 1
        ghi u vao thu_tu
        for v in ke[u] theo thu tu tang:
            if d[v] == -1:
                d[v] = d[u] + 1; cha[v] = u
                cuoi = cuoi + 1; hang[cuoi] = v
    return d, cha, thu_tu

KHOI_PHUC(cha, s, t, d):
    if d[t] == -1: return [], -1
    duong = []; v = t
    lap toi da n lan:
        them v vao duong
        if v == s: dao duong; return duong, d[t]
        v = cha[v]
    bao mang cha bi loi
```

2.3. Bất biến và tính tối ưu

Bất biến 1: d[v]≠−1 đúng khi v đã được phát hiện. Nguồn được đánh dấu trước vòng lặp; đỉnh khác chỉ đổi từ −1 một lần khi enqueue. Vì vậy không có bản sao trong hàng và cha của một đỉnh không bị ghi đè.

Bất biến 2: các đỉnh trong hàng được sắp theo khoảng cách không giảm, và chỉ có thể nằm ở hai mức liên tiếp. Khi lấy u ở đầu mức k, mọi đỉnh mới từ u nhận mức k+1

và được thêm cuối hàng. Chúng đứng sau các đỉnh mức k còn chờ; do đó FIFO duy trì bất biến.

Chứng minh d là ngắn nhất bằng quy nạp theo mức. Nguồn có độ dài 0. Giả sử các đỉnh mức k có khoảng cách tối ưu. Một v lần đầu phát hiện từ u mức k có tuyến dài k+1. Nếu tồn tại tuyến ngắn hơn, đỉnh đứng ngay trước v trên tuyến đó có khoảng cách $\leq k-1$ và đã được xử lý trước u; nó phải phát hiện v sớm hơn. Điều này mâu thuẫn giả thiết v còn chưa phát hiện. Vậy $d[v]=k+1$ là tối ưu.

Mọi đỉnh tới được đều được phát hiện: trên một tuyến từ s, mỗi đỉnh tiếp theo sẽ được xét khi đỉnh trước được lấy khỏi hàng. Khi hàng rỗng, không còn đỉnh tới được nào chưa xử lý. Do đó $d[t]=-1$ chính xác là không có tuyến tới t.

2.4. Tính đúng và kết thúc của truy vết

Với $v \neq s$ đã phát hiện, $cha[v]=u$ được gán tại một cạnh (u,v) và $d[u]=d[v]-1$. Theo cha làm khoảng cách giảm nghiêm ngặt cho đến 0 tại s; không có chu trình. Dãy ngược dài $d[t]+1 \leq n$ đỉnh. Sau đảo, đầu là s, cuối là t và các cặp liên tiếp là cạnh thật; số cạnh bằng d[t].

Hàm truy_vet còn chặn n bước cho dữ liệu cha do người gọi truyền sai. Giá trị 0 trước khi tới s trả đường rỗng; nhãn cha ngoài miền hoặc chu trình gây ArgumentError. Các guard này bảo vệ helper, không thay thế chứng minh rằng cha do BFS tạo ra là đúng.

BFS kết thúc sau tối đa n lần lấy đỉnh và hữu hạn lần quét kề. Truy vết kết thúc sau tối đa n bước. Bất biến đánh dấu lúc enqueue đồng thời ngăn tràn mảng hàng và duyệt lặp do chu trình.

2.5. Độ phức tạp theo từng công đoạn

Bảng 2.1. Chi phí; m là số cạnh vô hướng

Công đoạn	Thời gian	Bộ nhớ / điều kiện
Dựng kề + Set chống trùng	Kỳ vọng $O(n+m)$	$O(n+m)$; bảng băm không cam kết tệ nhất $O(1)$
Sắp từng ke[v]	$O(\Sigma \deg(v) \log \deg(v))$	Tính riêng trước BFS
BFS trên kề đã chuẩn bị	$O(n+m)$	$O(n)$ phụ, kể cả d/cha/hang/thứ tự
Truy vết l đỉnh	$O(l)$, $l \leq n$	$O(l)$ kết quả

Công đoạn	Thời gian	Bộ nhớ / điều kiện
Kiểm tuyến BFS đơn	O(n+m) chặn trên	Tìm thành viên trong kề là tuyến tính
Toàn quy trình tim_duong	Kỳ vọng O(n+m+Σ deg log deg)	Tổng lưu đồ thị và phụ O(n+m)

kiem_duong dùng phép “v in ke[u]” có thể quét deg(u) mục, nên không được ghi O(l) một cách tổng quát. Tuyến BFS là đơn; tổng bậc các đỉnh của tuyến không vượt tổng bậc toàn đồ thị 2m. Cộng kiểm nhãn và tuyến cho chặn O(n+m). Nếu truyền một walk tùy ý có l đỉnh lặp, chi phí có thể O(l·Δ).

Giới hạn $m \le n(n-1)/2$ cho O(m log n) ở khâu sắp trong đồ thị đơn. Khi kề đã tăng dần sẵn, lõi tìm đường chỉ cần O(n+m). Báo cáo không gộp sắp và bảng băm vào một tuyên bố tệ nhất O(n+m) cho toàn chương trình.

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT

3.1. Kiến trúc tệp và luồng thực hiện



Hình 3.1. Quy trình chuẩn bị kề, BFS, truy vết và đối chiếu

Bảng 3.1. Cấu trúc mã nguồn phát hành

Tệp	Trách nhiệm
src/BFS.jl	Năm hàm lõi; không truy cập tệp dữ liệu
demo.jl	In kết quả 8 demo hoặc đọc stdin --nhap
test/runtests.jl	59 phép assert, gồm chức năng và validator
test/verify_exhaustive.jl	Oracle Floyd–Warshall độc lập
scripts/test_catalog.jl	Bảng input/expected/actual/PASS của 28 ca
scripts/trace_and_scales.jl	Tái hiện trace T01, đối chiếu kết quả lõi; 9 khảo sát
Run_all.ps1	Chạy toàn bộ bằng một lệnh, dừng nếu thất bại
docs/logs và docs/anh_ket_qua	Log gốc và ảnh hiển thị log; không phải đầu vào

3.2. Hợp đồng năm hàm chính

Bảng 3.2. Đối số, tiền điều kiện và giá trị trả về

Hàm	Hợp đồng
tao_ke(n, canh)	Đồ thị đơn vô hướng theo miền Ch.1; trả Vector các Vector Int đã tăng dần; từ chối cạnh sai/trùng.
bfs_cha(ke,s)	ke do tao_ke tạo; s hợp lệ. Trả d,cha,thu_tu,so_phat_hien,so_xet_canh. Không sửa ke.

Hàm	Hợp đồng
truy_vet(cha,s,t)	Nhãn trong miền; trả tuyến thuận hoặc []; từ chối cha ngoài miền/chu trình.
kiem_duong(ke,duong)	Tuyến nhãn nguyên; kiểm không rỗng, nhãn và cạnh; trả Bool. Không tự chứng minh ngắn nhất.
tim_duong(n,canh,s,t)	Ghép các bước; trả cả kề, BFS, tuyến, số cạnh và số đỉnh truy vết; mỗi lần gọi cấp trạng thái mới.

3.3. Những dòng quyết định tính đúng

```
if d[v] == -1
    d[v] = d[u] + 1
    cha[v] = u
    cuoi += 1
    hang[cuoi] = v
end
```

Điều kiện chưa phát hiện dùng d[v]==−1, không dùng cha[v]==0 vì nguồn cũng có cha bằng 0. Gán d trước khi thêm hàng giúp các cạnh khác gặp v trong cùng mức không enqueue trùng. Giữ cha lần đầu là điều kiện để truy vết có cùng độ dài với d.

Sau mỗi lượt lấy u, biến dau tăng một đơn vị. so_xet_canh tăng trên mỗi mục kề, kể cả khi láng giềng đã phát hiện; vì vậy nó không phải số lần gán cha. so_phat_hien bằng số đỉnh đã lấy khỏi hàng do BFS chạy hết thành phần.

3.4. Truy vết từng bước ví dụ hình thoi

Bảng 3.3. Trạng thái sau khi xử lý xong u; thứ tự đỉnh trong d và cha là 1..4

Bước/u	dau/cuoi	Hàng chờ	d	cha
0 / –	1 / 1	[1]	[0,−1,−1,−1]	[0,0,0,0]
1 / 1	2 / 3	[2,3]	[0,1,1,−1]	[0,1,1,0]
2 / 2	3 / 4	[3,4]	[0,1,1,2]	[0,1,1,2]
3 / 3	4 / 4	[4]	[0,1,1,2]	[0,1,1,2]
4 / 4	5 / 4	[]	[0,1,1,2]	[0,1,1,2]

Tại bước 2, cha[4]=2. Bước 3 không đổi cha[4] dù cạnh (3,4) cũng tạo đường dài 2. Kết thúc hàng rỗng vì 5>4, dù vùng nhớ hang vẫn chứa [1,2,3,4]. Truy ngược từ t=4 qua 2 về 1, đảo thành [1,2,4].

Bảng được tái hiện bằng scripts/trace_and_scales.jl rồi assert d/cha cuối trùng kết quả hàm chính. Đây là script mô phỏng trạng thái để giải thích, không phải bộ gắn đo từng dòng trong src/BFS.jl. Giá trị đầy đủ nằm ở docs/logs/trace_scales.log.

3.5. Nhập bàn phím và xử lý dữ liệu sai

```
julia --startup-file=no demo.jl --nhap
# Lan luot go cac dong sau theo loi nhac:
4 4
1 2
1 3
2 4
3 4
1 4
```

Kết quả nhập bàn phím là tuyến 1→2→4, 2 cạnh, d=[0,1,1,2] và cha=[0,1,1,2]. Nhật ký keyboard.log chứng minh đã chạy luồng này, không chỉ chạy các hằng số demo.

Ví dụ sai n=3, các cạnh (1,2),(2,1) bị từ chối bằng ArgumentError “canh bi lap”. Demo 8 chỉ bắt đúng lỗi dự kiến, in thông báo đạt rồi trả mã 0; các lỗi khác bị ném lại. Không hiểu mã thoát 0 của demo này là lõi đã chấp nhận cạnh trùng.

Ký tự không nguyên hoặc thiếu số ở chế độ --nhap sẽ gây lỗi parse hoặc lỗi số trường. Phiên bản này không có vòng lặp hỏi lại; người dùng sửa đầu vào và chạy lại. Hàm chính kiểm miền của dữ liệu đã parse.

CHƯƠNG 4. KIỂM THỬ VÀ KẾT QUẢ

4.1. Môi trường, phiên bản và phương pháp

Môi trường kiểm chứng: Windows, PowerShell, Julia 1.13.1, thư viện chuẩn Test và Printf; không cài gói thuật toán ngoài. Tất cả lệnh chạy với --startup-file=no để tránh tác động startup.jl của người dùng. Ngày kiểm chứng 17/10/2026.

Commit C của nguồn dùng tạo kết quả:

```
af228fa7ea70b19bd801b7493326bfe2e76a3d75
```

Có ba tầng kiểm: các ca cụ thể có expected viết độc lập; kiểm thuộc tính đường/cha; oracle Floyd–Warshall trên mọi đồ thị đơn nhỏ. Độ dài tối ưu được đối chiếu qua oracle độc lập, không dùng lại BFS để tự tạo expected cho BFS.

Hai con số cần phân biệt: 28 ca có định danh gồm 12 chức năng và 16 helper/validator; trong test/runtests.jl có 59 phép assert vì một ca có thể kiểm nhiều trường. 26.705 cặp oracle thuộc lượt kiểm khác, không cộng thành “59 ca”.

4.2. Bảng kiểm 12 ca chức năng

Bảng 4.1. Expected và actual đã chạy; ký hiệu d là số cạnh tới đích

Mã	Đầu vào	Kỳ vọng	Thực tế / kết luận
T01	n=4;edges=[(3, 4), (1, 3), (2, 4), (1, 2)];s=1;t=4	path=[1, 2, 4];distance=2	path=[1, 2, 4];distance=2; PASS
T02	n=3;edges=[(1, 2)];s=1;t=3	path=Int64[];distance=-1	path=Int64[];distance=-1; PASS
T03	n=4;edges=[(1, 2), (1, 3), (2, 4), (3, 4)];s=2;t=2	path=[2];distance=0	path=[2];distance=0; PASS
T04	n=2;edges=[(1, 2)];s=2;t=1	path=[2, 1];distance=1	path=[2, 1];distance=1; PASS
T05	n=4;edges=[(1, 2), (2, 3), (3, 4)];s=1;t=4	path=[1, 2, 3, 4];distance=3	path=[1, 2, 3, 4];distance=3; PASS
T06	n=4;edges=[(1, 2), (2, 3), (3, 1), (3, 4)];s=1;t=4	path=[1, 3, 4];distance=2	path=[1, 3, 4];distance=2; PASS
T07	n=4;edges=[(2, 3), (3, 4)];s=1;t=4	path=Int64[];distance=-1	path=Int64[];distance=-1; PASS
T08	n=1;edges=Tuple{Int64, Int64}[];s=1;t=1	path=[1];distance=0	path=[1];distance=0; PASS
T09	n=5;edges=[(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5)];s=5;t=1	path=[5, 1];distance=1	path=[5, 1];distance=1; PASS

Mã	Đầu vào	Kỳ vọng	Thực tế / kết luận
T10	n=6;edges=[(1, 2), (2, 3), (4, 5), (5, 6)];s=2;t=5	path=Int64[];distance=-1	path=Int64[];distance=-1; PASS
T11	n=100; chain (1,2)..(99,100); s=100;t=1	path=[100,99,...,1];distance=99	path=100..1; vertices=100;distance=99; PASS
T12	n=5;edges=[(1, 5), (1, 3), (1, 4), (1, 2)];s=5;t=4	path=[5, 1, 4];distance=2	path=[5, 1, 4];distance=2; PASS

Các dòng dài được ghi đầy đủ trong scripts/test_catalog.jl và docs/logs/catalog.log. T11 ghi gọn dãy 100..1; mã kiểm so sánh toàn Vector với collect(100:-1:1), không chỉ so chuỗi tóm tắt.

4.3. Mười sáu ca kiểm helper và dữ liệu vào

Bảng 4.2. Ca V01–V16; actual tóm tắt sau khi predicate đã xác nhận

Mã	Đầu vào / lời gọi	Kỳ vọng	Thực tế
V01	tao_ke(0,[])	ArgumentError	ArgumentError / PASS
V02	tao_ke(101,[])	ArgumentError	ArgumentError / PASS
V03	n3;edge(1,4)	ArgumentError	ArgumentError / PASS
V04	n3;edge(0,2)	ArgumentError	ArgumentError / PASS
V05	n3;edge(1.0,2)	ArgumentError	ArgumentError / PASS
V06	n3;edge(1,1)	ArgumentError	ArgumentError / PASS
V07	n3;edges(1,2),(2,1)	ArgumentError	ArgumentError / PASS
V08	n3;edge12;s0;t2	ArgumentError	ArgumentError / PASS
V09	n3;edge12;s1;t4	ArgumentError	ArgumentError / PASS
V10	n3;edge12;path[1,3]	false	false / PASS
V11	n3;edge12;path[]	false	false / PASS
V12	n3;edge12;path[0]	false	false / PASS
V13	cha[2,1,0];s3;t1	ArgumentError	ArgumentError / PASS
V14	cha[0,4,0];s1;t2	ArgumentError	ArgumentError / PASS

Mã	Đầu vào / lời gọi	Kỳ vọng	Thực tế
V15	cha[0,1,0];s0;t2	ArgumentError	ArgumentError / PASS
V16	cha[0,1,0];s1;t3	[]	[] / PASS

Catalog bắt ngoại lệ và kiểm loại/thông điệp ở nơi được yêu cầu, hoặc kiểm kết quả Bool/Vector, rồi mới in nhãn actual. Nhãn ArgumentError trong bảng là tóm tắt kết quả predicate đã đạt; không nhận là bản chép stack trace đầy đủ.

Các ca helper không thay thế ca bài toán. V13 dùng cha có chu trình, V14 dùng cha ngoài miền, V16 cho đích không có cha. Chúng kiểm cơ chế bảo vệ của truy_vet, dù BFS đúng không sinh những trạng thái cha lỗi này.

4.4. Tám lần thực thi với kết quả cụ thể

T01 — hai tuyến cùng độ dài

```
julia --startup-file=no demo.jl --ca=1
```

T01 Hai đường cùng ít cạnh nhất

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=1
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=4; canh=[(1, 2), (1, 3), (2, 4), (3, 4)]; s=1; t=4
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |              0 |        0
   2 |              1 |        1
   3 |              1 |        1
   4 |              2 |        2
Thu tu BFS: 1 -> 2 -> 3 -> 4
Duong di: 1 -> 2 -> 4
So canh: 2
Kiem tung cap canh: DUNG
So dinh phat hien: 4
So muc ke da xet: 8
So dinh khi truy vet: 3
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.1. Ảnh hiển thị log thực chạy T01 — hai tuyến cùng độ dài

Đỉnh 2 được lấy trước 3; cha[4]=2, chọn [1,2,4], 2 cạnh. Tất cả 4 đỉnh được phát hiện; 8 mục kề được xét.

T02 — đích không tới được

```
julia --startup-file=no demo.jl --ca=2
```

T02 Đích không đến được

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=2
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=3; canh=[(1, 2)]; s=1; t=3
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |              0 |        0
   2 |              1 |        1
   3 |             -1 |        0
Thu tu BFS: 1 -> 2
KHONG CO DUONG
So dinh phat hien: 2
So muc ke da xet: 2
So dinh khi truy vet: 0
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.2. Ảnh hiển thị log thực chạy T02 — đích không tới được

Chỉ 1,2 thuộc thành phần nguồn. d[3]=−1; tuyến rỗng. Đây là ví dụ 2 trong phiếu.

T03 — nguồn trùng đích

```
julia --startup-file=no demo.jl --ca=3
```

T03 Nguồn trùng đích

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=3
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=4; canh=[(1, 2), (1, 3), (2, 4), (3, 4)]; s=2; t=2
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |              1 |        2
   2 |              0 |        0
   3 |              2 |        1
   4 |              1 |        2
Thu tu BFS: 2 -> 1 -> 4 -> 3
Duong di: 2
So canh: 0
Kiem tung cap canh: DUNG
So dinh phat hien: 4
So muc ke da xet: 8
So dinh khi truy vet: 1
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.3. Ảnh hiển thị log thực chạy T03 — nguồn trùng đích

Với s=t=2, tuyến [2], 0 cạnh; bảng d vẫn bao phủ thành phần vì BFS không dừng sớm.

T07 — nguồn cô lập

```
julia --startup-file=no demo.jl --ca=4
```

T07 Nguồn cô lập

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=4
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=4; canh=[(2, 3), (3, 4)]; s=1; t=4
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |             0 |        0
   2 |            -1 |        0
   3 |            -1 |        0
   4 |            -1 |        0
Thu tu BFS: 1
KHONG CO DUONG
So dinh phat hien: 1
So muc ke da xet: 0
So dinh khi truy vet: 0
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.4. Ảnh hiển thị log thực chạy T07 — nguồn cô lập

Nguồn 1 không có cạnh; chỉ phát hiện 1, không xét mục kề. Thành phần 2,3,4 không được đi xuyên sang.

T05 — chuỗi phải truy vết dài

```
julia --startup-file=no demo.jl --ca=5
```

T05 Chuỗi bốn đỉnh

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=5
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=4; canh=[(1, 2), (2, 3), (3, 4)]; s=1; t=4
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |              0 |        0
   2 |              1 |        1
   3 |              2 |        2
   4 |              3 |        3
Thu tu BFS: 1 -> 2 -> 3 -> 4
Duong di: 1 -> 2 -> 3 -> 4
So canh: 3
Kiem tung cap canh: DUNG
So dinh phat hien: 4
So muc ke da xet: 6
So dinh khi truy vet: 4
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.5. Ảnh hiển thị log thực chạy T05 — chuỗi phải truy vết dài

Theo cha 4→3→2→1 rồi đảo; đường [1,2,3,4], 3 cạnh. Phân biệt số đỉnh 4 với số cạnh 3.

T06 — chu trình

```
julia --startup-file=no demo.jl --ca=6
```

T06 Đồ thị có chu trình

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=6
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=4; canh=[(1, 2), (2, 3), (3, 1), (3, 4)]; s=1; t=4
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |              0 |        0
   2 |              1 |        1
   3 |              1 |        1
   4 |              2 |        3
Thu tu BFS: 1 -> 2 -> 3 -> 4
Duong di: 1 -> 3 -> 4
So canh: 2
Kiem tung cap canh: DUNG
So dinh phat hien: 4
So muc ke da xet: 8
So dinh khi truy vet: 3
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.6. Ảnh hiển thị log thực chạy T06 — chu trình

Cạnh 1–3 cho đường [1,3,4], 2 cạnh. Đánh dấu khi enqueue tránh đưa lại 1 hoặc 3.

T08 — miền nhỏ nhất

```
julia --startup-file=no demo.jl --ca=7
```

T08 Một đỉnh không cạnh

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=7
DT076 - TIM DUONG DI IT CANH NHAT BANG BFS
n=1; canh=Tuple{Int64, Int64}[]; s=1; t=1
Lang gieng tang dan; cha chi gan khi phat hien lan dau.
Dinh | Khoang cach d | Dinh cha
   1 |             0 |        0
Thu tu BFS: 1
Duong di: 1
So canh: 0
Kiem tung cap canh: DUNG
So dinh phat hien: 1
So muc ke da xet: 0
So dinh khi truy vet: 1
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.7. Ảnh hiển thị log thực chạy T08 — miền nhỏ nhất

n=1, m=0, s=t=1; tuyến [1], 0 cạnh và 0 mục kề. Không cần có cạnh để tuyến nguồn=đích hợp lệ.

V07 — cạnh vô hướng bị nhập lặp

```
julia --startup-file=no demo.jl --ca=8
```

V07 Từ chối cạnh trùng

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no demo.jl --ca=8
DT076 - MINH HOA TU CHOI DU LIEU KHONG HOP LE
n=3; canh=[(1, 2), (2, 1)]; s=1; t=3
Mong doi: canh (2, 1) lap lai canh vo huong (1, 2).
DA TU CHOI DU LIEU: ArgumentError: canh bi lap
KET QUA MINH HOA: DUNG; loi du kien da duoc bat.
Ma thoat 0 chi danh cho demo bat loi dung nhu mong doi.
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.8. Ảnh hiển thị log thực chạy V07 — cạnh vô hướng bị nhập lặp

(2,1) trùng (1,2), bị từ chối. Demo bắt đúng lỗi dự kiến; không đưa input sai vào BFS.

4.5. Tổng kết test và đối chiếu độc lập

Bộ kiểm thử 59 phép kiểm

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no test/runtests.jl
Test Summary:           | Pass  Total  Time
DT076 - 12 ca chuc nang |   43     43  0.3s
Test Summary:                 | Pass  Total  Time
Du lieu vao va ham kiem duong |   16     16  0.1s
DA QUA 12 CA CHUC NANG VA CAC KIEM TRA DU LIEU VAO.
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.9. Kết quả thực chạy 43+16=59 phép assert

test/runtests.jl đạt 43/43 kiểm tra chức năng và 16/16 kiểm tra dữ liệu vào/hàm phụ. Cả hai testset đều hoàn thành, mã thoát 0. Con số thời gian trong log là thời gian của lượt chạy cụ thể, không dùng làm benchmark.

Oracle Floyd–Warshall độc lập

```
$ julia --version
julia version 1.13.1
$ julia --startup-file=no test/verify_exhaustive.jl
EXHAUSTIVE_OK graphs=1099 source_target_pairs=26705 n=1..5
ORACLE=Floyd-Warshall; distances, endpoints, real edges, simple path
and parent distances checked
This checks finite graphs up to n=5; mathematical proof covers all
valid n.
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.10. Oracle Floyd–Warshall trên toàn bộ đồ thị đơn n≤5

Với mỗi n=1..5, liệt kê mọi tập con của các cặp u<v, tổng Σ2^(n(n−1)/2)=1+2+8+64+1024=1.099 đồ thị. Mỗi đồ thị kiểm n^2 cặp nguồn/đích, tổng 1+8+72+1024+25600=26.705 cặp.

Oracle khởi tạo khoảng cách ma trận rồi chạy Floyd–Warshall $O(n^3)$, không gọi BFS để lập expected. Với mỗi nguồn, kiểm d ở tất cả đỉnh. Với đích tới được, kiểm tuyến đầu/cuối, số cạnh bằng khoảng cách oracle, không lặp đỉnh, từng cạnh thật và cha giảm d một đơn vị. Đích không tới thì tuyến rỗng và d=−1.

Kết quả EXHAUSTIVE_OK xác nhận toàn bộ tập hữu hạn n≤5 đã đạt. Nó phát hiện nhiều lỗi cài đặt và quy tắc cha, nhưng không là phép duyệt toàn miền n≤100. Tính đúng tổng quát vẫn dựa vào chứng minh Chương 2 và các ca biên n=100.

4.6. Khảo sát số thao tác theo quy mô

Bảng 4.3. Bộ đếm thực chạy; BFS duyệt hết thành phần, nguồn 1, đích n

Họ	n	m	Phát hiện	Mục kề	Đỉnh tuyến	d[t]
Chuỗi	10	9	10	18	10	9
Sao	10	9	10	18	2	1
Đầy đủ	10	45	10	90	2	1
Chuỗi	30	29	30	58	30	29
Sao	30	29	30	58	2	1
Đầy đủ	30	435	30	870	2	1
Chuỗi	100	99	100	198	100	99
Sao	100	99	100	198	2	1
Đầy đủ	100	4950	100	9900	2	1

Cả ba họ đều liên thông nên phát hiện n đỉnh và xét 2m mục kề. Chuỗi/sao có m=n−1 nên số mục kề tăng tuyến tính theo n. Đầy đủ có m=n(n−1)/2 nên số mục kề=n(n−1), tăng bậc hai theo n và vẫn phù hợp O(n+m).

Đường chuỗi có n đỉnh trong truy vết và d[t]=n−1; sao/đầy đủ ở cấu hình này có 2 đỉnh và d[t]=1. Dù đã phát hiện đích sớm, chương trình tiếp tục BFS toàn thành phần, giải thích vì sao đồ thị đầy đủ vẫn xét 9.900 mục kề khi n=100.

Các giá trị là số thao tác do chương trình đếm, không là mili giây và không bao gồm số so sánh của sort!/Set. Không dùng chúng để tuyên bố đã đo thời gian toàn quy trình hay xác nhận độ phức tạp chỉ bằng ba kích cỡ.

4.7. Chạy lại từ clone sạch và truy nguyên phiên bản

Clone kho local và chạy toàn bộ

```
$ git clone <kho_ma_cuc_bo> clone_source_C
CLONE_LOCAL_OK
$ git rev-parse HEAD
af228fa7ea70b19bd801b7493326bfe2e76a3d75
$ .\Run_all.ps1
julia version 1.13.1
Test Summary:           | Pass  Total  Time
DT076 - 12 ca chuc nang |   43     43  0.3s
Test Summary:                 | Pass  Total  Time
Du lieu vao va ham kiem duong |   16     16  0.1s
DA QUA 12 CA CHUC NANG VA CAC KIEM TRA DU LIEU VAO.
EXHAUSTIVE_OK graphs=1099 source_target_pairs=26705 n=1..5
CATALOG_OK functional_cases=12 helper_validator_cases=16
total_cases=28
END_TO_END_OK: demos, tests, independent oracle, trace and scale
counters completed.
[EXIT CODE: 0]
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.11. Clone cục bộ mới tại C rồi chạy toàn bộ Run_all.ps1

Đã clone repository local sang một thư mục mới, xác nhận HEAD bằng C, rồi chạy Run_all.ps1. Các demo, 59 assert, oracle, catalog và trace/scale đều hoàn tất; END_TO_END_OK và mã thoát 0. Hình là log rút các dòng tổng kết; clone_all.log giữ toàn bộ stdout.

Kho nguồn có README, mã, test, scripts và .gitignore được commit thật. Hình tiếp theo ghi Git log, HEAD, trạng thái sạch và các tệp theo dõi tại C. Danh tính commit

"CTDLGT Reference Build" là danh tính bản dựng cục bộ, không đại diện cho sinh viên.

Git commit của nguồn đã chạy

```
$ git rev-parse HEAD
af228fa7ea70b19bd801b7493326bfe2e76a3d75
$ git log -1 --oneline
af228fa Complete DT076 BFS source tests independent oracle and
reproduction scripts
$ git status --short

WORKTREE_CLEAN
$ git ls-files
.gitignore
README.md
Run_all.ps1
demo.jl
docs/anh_ket_qua/README.md
scripts/test_catalog.jl
scripts/trace_and_scales.jl
src/BFS.jl
test/runtests.jl
test/verify_exhaustive.jl
```

Bản hiển thị stdout thực của bản giải ngày 17/10/2026. Log đầy đủ đi kèm mã nguồn.

Hình 4.12. Git commit nguồn C và trạng thái kho đã kiểm

SHA-256 của src/BFS.jl:

`f6a5358f0c11148fcf9c0dee2f5c03f855ccc0723dca11103684b8e811a6009d`

Commit C chỉ đến phiên bản nguồn đã chạy. Commit đóng gói sau đó có thể bổ sung ảnh và log mà không đổi mã. Git bundle phát kèm giữ lịch sử để clone ngoại tuyến. Thông tin commit đóng gói được ghi trong PHIEN_BAN.txt, tránh nhầm SHA báo cáo với HEAD sau khi thêm bằng chứng.

Bản mẫu chưa công bố một kho GitHub cá nhân. Minh chứng local hoàn tất luồng kiểm kỹ thuật từ commit tới clone/chạy lại; sinh viên khi nộp cần thêm repo GitHub thật của mình theo Classroom. Không dùng local path thay URL GitHub trong bài nộp.

CHƯƠNG 5. ĐÁNH GIÁ VÀ TÁI HIỆN

5.1. Đối chiếu nhiệm vụ với sản phẩm

Bảng 5.1. Kết quả nghiệm thu có thể kiểm lại

Yêu cầu	Sản phẩm / kết quả
Đúng hợp đồng và hai ví dụ	Ch.1; T01,T02 đúng expected
Tự cài cấu trúc và thuật toán	src/BFS.jl; FIFO mảng; d/cha lần đầu
Giải thích đầy đủ	Ch.2 chứng minh; Ch.3 bảng trace
Ca thường, biên và sai	12 chức năng +16 validator; 59 assert đạt
Đối chiếu độc lập	Floyd: 1.099 đồ thị /26.705 cặp đạt
Khảo sát ba quy mô	n=10,30,100 cho chuỗi/sao/đầy đủ
Chạy lại từ nguồn	Clone C thực tế; toàn bộ quy trình exit 0
Tài liệu và bằng chứng	12 hình log, log thô, Word/PDF, mã và bundle

5.2. Giới hạn và hướng phát triển

Lời giải dùng cho đồ thị không trọng số. Khi cạnh có trọng số khác nhau, ít cạnh nhất không đồng nghĩa tổng trọng số nhỏ nhất; cần thuật toán phù hợp như Dijkstra trong điều kiện của đề tương ứng. Không mở rộng âm thầm bài toán hiện tại.

Miền $n \leq 100$ và đồ thị đơn giúp dễ quan sát. Hỗ trợ đồ thị lớn cần xem lại giới hạn bộ nhớ, cách nhập và xử lý cạnh. Hiện chương trình báo lỗi khi nhập sai và dừng, chưa có giao diện hỏi lại; không coi đây là ứng dụng quản lý hoàn chỉnh.

Có thể thêm chức năng in mọi đường ngắn nhất bằng lưu nhiều tiền nhiệm, nhưng số đường có thể rất lớn. Bản này chỉ cần một đường theo thứ tự đã quy định. Mọi mở rộng phải giữ test hợp đồng hiện có và phân tích lại chi phí.

5.3. Các câu hỏi bảo vệ đã có lời giải

Vì sao không đánh dấu lúc lấy khỏi hàng?

Một đỉnh có thể được nhiều cha enqueue trước lần lấy đầu tiên, gây trùng và có nguy cơ vượt hàng n ô. Đánh dấu lúc enqueue tránh điều này.

Vì sao cha[4] không đổi ở cạnh 3–4?

Lần đầu từ 2 đã cho khoảng cách tối ưu. Đổi cha sau có thể phá quy tắc phá hòa và tính nhất quán trace.

Đếm số cạnh hay số đỉnh trên tuyến?

Tuyến l đỉnh có l−1 cạnh; s=t có 1 đỉnh nhưng 0 cạnh.

BFS có dừng ngay khi thấy t không?

Phiên bản này không dừng sớm để trả bảng toàn thành phần và bộ đếm rõ nghĩa. Bản dừng sớm phải đổi hợp đồng đầu ra.

Kiểm từng cặp cạnh có chứng minh tối ưu không?

Chỉ chứng minh hợp lệ; tối ưu dựa chứng minh FIFO và đối chiếu Floyd độc lập.

Chạy hết n≤5 có đảm bảo không lỗi mọi n không?

Đó là bằng chứng hữu hạn; ca n=100 và chứng minh tổng quát bổ sung cho giới hạn của thử nghiệm.

5.4. Thực hiện lại từ gói đã tải

Giải nén toàn bộ gói vào thư mục có quyền ghi. Mở Ma_BFS_mau hoặc giải nén Ma_BFS_mau.zip. Cài Julia; nếu julia chưa có trên PATH, truyền đường dẫn thực đến executable qua tham số -Julia. Chương trình chỉ cần Julia và thư viện chuẩn.

```
.\Run_all.ps1
# Hoac khi Julia chua co tren PATH:
.\Run_all.ps1 -Julia 'DUONG_DAN_THUC_DEN_JULIA_EXE'
# Kiem tra rieng luong nhap:
julia --startup-file=no demo.jl --nhap
```

Để kiểm lịch sử và clone ngoại tuyến, chạy từ thư mục có BFS_mau.bundle:

```
git -c core.autocrlf=false clone BFS_mau.bundle BFS_clone
cd BFS_clone
git rev-parse HEAD
git show af228fa7ea70:src/BFS.jl
.\Run_all.ps1
```

HEAD của bundle có thể là commit bổ sung ảnh/log; commit C vẫn tồn tại trong lịch sử. Các hash ở phụ lục tính byte của tệp trong ZIP phát hành. Git chuẩn hóa xuống dòng của một số tệp CRLF tại commit; tham số core.autocrlf=false giữ LF trong bản clone. Khi đối chiếu ZIP với Git, chuẩn hóa CRLF thành LF trước khi so nội dung; riêng src/BFS.jl vốn LF nên hash không đổi. Nếu cần tái hiện chính xác C, dùng git checkout --detach af228fa7ea70 sau khi clone rồi chạy Run_all.ps1.

Với bài nộp của sinh viên, giữ bốn sản phẩm DOCX, PDF, MaNguon.zip, ThongTinNop.txt; tên theo MSSV_DTxxx_HoTenKhongDau. Báo cáo Word/PDF khớp nhau, mã ZIP đầy đủ source/test/README/ảnh, thông tin nộp ghi URL GitHub và commit đầy đủ đã chạy. Hạn nộp lấy từ bài giao trên Classroom.

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật, INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf do giảng viên cung cấp; mục 6.2, 6.3.2, Bài tập 28; trang in 140–145,159. Không gán thêm tác giả/năm khi dữ liệu nguồn chưa xác minh.

[2] Đề cương môn học trong bộ tài liệu học phần và danh sách INT13185 nhóm 03. Dùng để xác định phạm vi nội dung và phân công; hai mã được giữ đúng nguồn.

[3] MAU TRINH BAY/Mau_bia_va_bao_cao_CTDL_GT.docx, mẫu hình thức giảng viên cung cấp. A4, Times New Roman, hệ bìa/mục đầu/nội dung/phụ lục kế thừa từ mẫu.

[4] Mã tham khảo DT076, commit C: af228fa7ea70b19bd801b7493326bfe2e76a3d75. Gồm src/BFS.jl, demo.jl, test và scripts; log và hash nằm trong gói này.

[5] GitHub Docs, Quickstart for repositories, https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories; tham khảo thao tác tạo kho cá nhân, truy cập 17/10/2026.

[6] GitHub Docs, Getting permanent links to files, https://docs.github.com/en/repositories/working-with-files/using-files/getting-permanent-links-to-files; tham khảo permalink gắn commit, truy cập 17/10/2026.

PHỤ LỤC A. TOÀN BỘ MÃ LỖI

Nội dung src/BFS.jl tại commit C; không lược bỏ phần xử lý đặc biệt. Tệp chạy được nằm trong Ma_BFS_mau.

```
module BFS

export tao_ke, bfs_cha, truy_vet, kiem_duong, tim_duong

"""Dựng danh sách kề của đồ thị vô hướng đơn; nhãn đỉnh là 1:n."""
function tao_ke(n::Integer, canh)
    1 <= n <= 100 || throw(ArgumentError("n phai nam trong 1..100"))
    ke = [Int[] for _ in 1:n]
    da_co = Set{Tuple{Int,Int}}()
    for (u, v) in canh
        (u isa Integer && v isa Integer) || throw(ArgumentError("nhan dinh phai
la so nguyen"))
        (1 <= u <= n && 1 <= v <= n) || throw(ArgumentError("dinh nam ngoai
1..n"))
        u != v || throw(ArgumentError("do thi don khong co canh khuyen"))
        e = minmax(Int(u), Int(v))
        e in da_co && throw(ArgumentError("canh bi lap"))
        push!(da_co, e)
        push!(ke[u], v)
        push!(ke[v], u)
    end
    # Thứ tự này quyết định đường được chọn khi có nhiều đường ngắn nhất.
    for ds in ke
        sort!(ds)
    end
    return ke
end

"""
    bfs_cha(ke, s)

Duyệt hết thành phần liên thông chứa s. d[v]=-1 nếu chưa tới được v;
cha[s]=0; cha[v] chỉ được gán ở lần đầu phát hiện. Hàng đợi dùng mảng
cấp phát trước, chỉ số dau và cuoi; không dùng popfirst!.
"""
function bfs_cha(ke, s::Integer)
    n = length(ke)
    1 <= s <= n || throw(ArgumentError("nguon nam ngoai 1..n"))
    d = fill(-1, n)
    cha = zeros(Int, n)
    hang = zeros(Int, n)
    thu_tu = Int[]
    dau, cuoi = 1, 1
    hang[cuoi] = s
    d[s] = 0
    so_xet_canh = 0

    while dau <= cuoi
        u = hang[dau]
        dau += 1
        push!(thu_tu, u)
        for v in ke[u]
```

```
            so_xet_canh += 1
            if d[v] == -1
                # Đánh dấu ngay khi đưa vào hàng để mỗi đỉnh chỉ vào một lần.
                d[v] = d[u] + 1
                cha[v] = u
                cuoi += 1
                hang[cuoi] = v
            end
        end
    end
    return (; d, cha, thu_tu, so_phat_hien=length(thu_tu), so_xet_canh)
end

"""Lần theo cha từ t về s rồi đảo; trả Int[] nếu không có đường."""
function truy_vet(cha, s::Integer, t::Integer)
    n = length(cha)
    (1 <= s <= n && 1 <= t <= n) || throw(ArgumentError("s, t nam ngoai 1..n"))
    duong = Int[]
    v = Int(t)
    # Tối đa n đỉnh; chặn cả trường hợp mảng cha do người gọi truyền bị lỗi.
    for _ in 1:n
        push!(duong, v)
        if v == s
            reverse!(duong)
            return duong
        end
        v = cha[v]
        v == 0 && return Int[]
        1 <= v <= n || throw(ArgumentError("mang cha khong hop le"))
    end
    throw(ArgumentError("mang cha co chu trinh"))
end

"""Kiểm tra nhãn đỉnh và từng cặp đỉnh liên tiếp trên một đường không rỗng."""
function kiem_duong(ke, duong)
    isempty(duong) && return false
    all(v -> 1 <= v <= length(ke), duong) || return false
    for i in 1:(length(duong) - 1)
        duong[i + 1] in ke[duong[i]] || return false
    end
    return true
end

"""Ghép dựng đồ thị, BFS và truy vết; không đọc bất kỳ tệp dữ liệu nào."""
function tim_duong(n::Integer, canh, s::Integer, t::Integer)
    ke = tao_ke(n, canh)
    1 <= t <= n || throw(ArgumentError("dich nam ngoai 1..n"))
    kq = bfs_cha(ke, s)
    duong = kq.d[t] == -1 ? Int[] : truy_vet(kq.cha, s, t)
    so_canh = kq.d[t]
    if !isempty(duong)
        @assert length(duong) - 1 == so_canh
        @assert kiem_duong(ke, duong)
    end
    return (; ke, kq..., duong, so_canh, so_buoc_truy_vet=length(duong))
end

end # module BFS
```

PHỤ LỤC B. DEMO VÀ NHẬP BÀN PHÍM

demo.jl gọi cùng hàm lõi, không có một BFS riêng chỉ để tạo hình đẹp. Demo 8 bắt đúng lỗi cạnh trùng dự kiến.

```
include(joinpath(@__DIR__, "src", "BFS.jl"))
using .BFS
using Printf

function in_ket_qua(n, canh, s, t)
    kq = tim_duong(n, canh, s, t)
    println("DT076 - TIM DUONG DI IT CANH NHAT BANG BFS")
    println("n=$n; canh=$canh; s=$s; t=$t")
    println("Lang gieng tang dan; cha chi gan khi phat hien lan dau.")
    println("Dinh | Khoang cach d | Dinh cha")
    for v in 1:n
        @printf("%4d | %13d | %8d\n", v, kq.d[v], kq.cha[v])
    end
    println("Thu tu BFS: ", join(kq.thu_tu, " -> "))
    if kq.so_canh == -1
        println("KHONG CO DUONG")
    else
        println("Duong di: ", join(kq.duong, " -> "))
        println("So canh: ", kq.so_canh)
        println("Kiem tung cap canh: ", kiem_duong(kq.ke, kq.duong) ? "DUNG" :
"SAI")
    end
    println("So dinh phat hien: ", kq.so_phat_hien)
    println("So muc ke da xet: ", kq.so_xet_canh)
    println("So dinh khi truy vet: ", kq.so_buoc_truy_vet)
    return kq
end

function nhap_ban_phim()
    println("Nhap n m (vi du: 4 4):")
    nm = parse.(Int, split(readline()))
    length(nm) == 2 || throw(ArgumentError("can dung 2 so n m"))
    n, m = nm
    1 <= n <= 100 || throw(ArgumentError("n phai nam trong 1..100"))
    0 <= m <= n * (n - 1) ÷ 2 || throw(ArgumentError("m khong hop le"))
    canh = Tuple{Int,Int}[]
    for i in 1:m
        println("Nhap canh $i (u v):")
        uv = parse.(Int, split(readline()))
        length(uv) == 2 || throw(ArgumentError("moi canh can dung 2 so"))
        push!(canh, (uv[1], uv[2]))
    end
    println("Nhap s t:")
    st = parse.(Int, split(readline()))
    length(st) == 2 || throw(ArgumentError("can dung 2 so s t"))
    in_ket_qua(n, canh, st[1], st[2])
end

function minh_hoa_du_lieu_sai()
    println("DT076 - MINH HOA TU CHOI DU LIEU KHONG HOP LE")
    println("n=3; canh=[(1, 2), (2, 1)]; s=1; t=3")
    println("Mong doi: canh (2, 1) lap lai canh vo huong (1, 2).")
```

```
    try
        tim_duong(3, [(1, 2), (2, 1)], 1, 3)
    catch loi
        if loi isa ArgumentError && occursin("canh bi lap", sprint(showerror,
loi))
            println("DA TU CHOI DU LIEU: ", sprint(showerror, loi))
            println("KET QUA MINH HOA: DUNG; loi du kien da duoc bat.")
            println("Ma thoat 0 chi danh cho demo bat loi dung nhu mong doi.")
            return nothing
        end
        rethrow()
    end
    error("LOI: chuong trinh da chap nhan canh lap")
end

if "--nhap" in ARGS
    nhap_ban_phim()
elseif any(startswith("--ca="), ARGS)
    ca = only(filter(startswith("--ca="), ARGS))
    if ca == "--ca=1"
        in_ket_qua(4, [(1, 2), (1, 3), (2, 4), (3, 4)], 1, 4)
    elseif ca == "--ca=2"
        in_ket_qua(3, [(1, 2)], 1, 3)
    elseif ca == "--ca=3"
        in_ket_qua(4, [(1, 2), (1, 3), (2, 4), (3, 4)], 2, 2)
    elseif ca == "--ca=4"
        in_ket_qua(4, [(2, 3), (3, 4)], 1, 4)
    elseif ca == "--ca=5"
        in_ket_qua(4, [(1, 2), (2, 3), (3, 4)], 1, 4)
    elseif ca == "--ca=6"
        in_ket_qua(4, [(1, 2), (2, 3), (3, 1), (3, 4)], 1, 4)
    elseif ca == "--ca=7"
        in_ket_qua(1, Tuple{Int,Int}[], 1, 1)
    elseif ca == "--ca=8"
        minh_hoa_du_lieu_sai()
    else
        throw(ArgumentError("chon --ca=1 den --ca=8"))
    end
else
    # Hai ví dụ đúng như phiếu DT076. Có thể sửa trực tiếp các biến này.
    in_ket_qua(4, [(1, 2), (1, 3), (2, 4), (3, 4)], 1, 4)
    println("\n--- VI DU 2: DICH KHONG TOI DUOC ---")
    in_ket_qua(3, [(1, 2)], 1, 3)
end
```

PHỤ LỤC C. CHẠY TỰ ĐỘNG VÀ TOÀN VẸN

```
param([string]$Julia='julia')
$ErrorActionPreference='Stop'
$codeRoot=$PSScriptRoot
function Run-JuliaStep([string[]]$JuliaArgs) {
    & $Julia @JuliaArgs
    if ($LASTEXITCODE -ne 0) { throw ('Julia failed: '+($JuliaArgs -join ' ')) }
}
Run-JuliaStep @('--version')
for ($caseId=1; $caseId -le 8; $caseId++) {
    Write-Output ('=== DEMO '+$caseId+' ===')
    Run-JuliaStep @('--startup-file=no',(Join-Path $codeRoot 'demo.jl'),('--
ca='+$caseId))
}
Run-JuliaStep @('--startup-file=no',(Join-Path $codeRoot 'test/runtests.jl'))
Run-JuliaStep @('--startup-file=no',(Join-Path $codeRoot
'test/verify_exhaustive.jl'))
Run-JuliaStep @('--startup-file=no',(Join-Path $codeRoot
'scripts/test_catalog.jl'))
Run-JuliaStep @('--startup-file=no',(Join-Path $codeRoot
'scripts/trace_and_scales.jl'))
Write-Output 'END_TO_END_OK: demos, tests, independent oracle, trace and scale
counters completed.'
```

Mã test đầy đủ được phát nguyên tệp test/runtests.jl và test/verify_exhaustive.jl; bảng kết quả đầy đủ ở catalog.log. Không lược bớt chúng trong gói mã. SHA-256 sau đây cho phép đối chiếu tệp đã dùng kiểm, kể cả khi tách khỏi Git.

Tệp	SHA-256
demo.jl	ebb0c5543172f7976fa62dc6094781c262a1a8db51bc3b7d655d2ecbdb59d037
Run_all.ps1	9fbd9d794d87dfaf27dd133243c307a07c2d056cf151748a434d2ff653e6a15e
scripts/test_catalog.jl	7d152cc8e7c931cc82f57b094e86bd859c50e3fc92546d98eff731e86f16af34
scripts/trace_and_scales.jl	3dbeb092d4b7341fd4484aef245159e1b89d365236f08655902c6cbfb87cd5ab
src/BFS.jl	f6a5358f0c11148fcf9c0dee2f5c03f855ccc0723dca11103684b8e811a6009d
test/runtests.jl	b2a99eaa24ba60fabd2131edbe36ddf3c58dacc97c11106311919955ea191d51
test/verify_exhaustive.jl	72b12a2d06edc2145d750ce11c59f3ea6eb1285dc3af8a30796b704f461e0f98

PHỤ LỤC D. PHIẾU TỰ KIỂM TRƯỚC NỘP

Mục	Kết quả mẫu	Sinh viên cần thực hiện
Nhận diện	Bản giảng dạy, không có MSSV giả	Điền đúng họ tên/MSSV/lớp/mã đề
Nội dung	Giải trọn DT076	Làm đúng phiếu cá nhân, xóa nội dung lệch đề
Dữ liệu	Bàn phím/khai báo trực tiếp	Mô tả miền và quy tắc, không phụ thuộc CSV
Bảng test	12+16 ca, 59 assert đạt	Expected tự tính, actual tự chạy, đủ ca giao
Ảnh	12 hình log tham khảo	Tự chụp 11 ảnh A01–A11; phân tích từng ảnh
Phiên bản	C và clone local có thật	URL GitHub thật, full commit, permalink, clone lại
Báo cáo	Word/PDF cùng nội dung	Cập nhật mục lục/số hình/bảng, xuất PDF mới
Nộp Classroom	Hướng dẫn trong gói	Bốn file đúng tên; đính kèm và bấm Nộp bài

Không dùng ảnh hoặc log mẫu làm bằng chứng chạy cá nhân. Khi chỉnh mã, phải chạy lại test/ảnh và ghi commit mới. Khi chỉ sửa chính tả báo cáo, giữ đúng commit mã được kiểm và nêu rõ phiên bản tài liệu.